

FUNDAÇÃO UNIVERSIDADE DO ESTADO DE SANTA CATARINA - UDESC
CENTRO DE CIÊNCIAS TECNOLÓGICAS - CCT

ANA LUIZA CAPRISTRANO DA CRUZ E GUILHERME HOERNING REINERT

**RELATÓRIO DA TAREFA 1 DE TEORIA DE GRAFOS:
IMPLEMENTAÇÃO DE UM GRAFO COM CARGA PRIMÁRIA DE ARQUIVOS CSV**

JOINVILLE

2026

Sumário

1. Objetivos da Tarefa.....	3
2. Arquivos CSV de Entrada.....	3
3. Estrutura do Código.....	4
3.1. Saída e Descrição do Grafo.....	4
3.2. Funções e Structs Criados.....	6
4. Dificuldades Encontradas.....	10
5. Resultados.....	11

1. Objetivos da Tarefa

Escrever um código na linguagem C capaz de extrair as os dados inteiros contidos em um arquivo de entrada CSV sob o formato de uma lista de adjacências. O código deverá então criar outra lista de adjacências para interpretar as principais características referentes ao grafo, bem como

1. Número total de vértices;
2. Grau máximo e grau mínimo;
3. Se ele é simples um um multigrafo;
4. Detalhar os componentes conexos existentes, caso houver mais de um, logo a distribuição dos vértices em cada componente, a sua quantidade e tamanhos.

O código deverá ser capaz de ser compilado no Ubuntu e organizado em uma pasta compactada, conforme as especificações de entrega no MOODLE.

2. Arquivos CSV de Entrada

Cada arquivo de entrada expressa uma lista de adjacências, ou seja, cada linha do arquivo corresponde a um vértice de um grafo e as suas respectivas adjacências ou conexões com outros vértices, formando assim todas as arestas do grafo.

O arquivo **teste2.csv** foi disponibilizado pelo professor como principal exemplo de input para o algoritmo, o qual possui 999 linhas com 999 vértices, sendo a primeira linha dedicada para valores de metadados: “100 2”, com 2 denunciando as 2 colunas para a leitura do arquivo, conforme a Figura 1.

Entretanto, o algoritmo foi criado esperando uma quantidade indeterminada de linhas, vértices e adjacências em um mesmo vértice, portanto qualquer número de colunas. Para tal foi também criado **teste1.csv** contendo a lista de adjacências de um grafo de 5 vértices, significativamente menor e também utilizado como material dedicado à disciplina.

```
teste2.csv > data
You, yesterday | 2 authors (guiHReinert and one other)
1 100 2
2 1 3
3 2 4
4 3 5
5 4 6
6 5 7
7 6 8
8 7 9
9 8 10
10 9 11
11 10 12
12 11 13
13 12 14
14 13 15
15 14 16
16 15 17
17 16 18
18 17 19
19 18 20
20 19 21
21 20 22
22 21 23
```

Figura 1: Parte inicial dos dados de teste2.csv.

3. Estrutura do Código

3.1. Saída e Descrição do Grafo

O arquivo principal de compilação do código é **main.c**. Nele é incluído o *header* principal **arq.h**, que contém todas as bibliotecas e funções utilizadas. Uma vez que a parte mais essencial do algoritmo se localiza em **arq.h**, este arquivo se concentra em declarar as variáveis iniciais e as funções utilizadas para gerar o *output* e detalhamento do grafo. Há também comentários referentes à compilação e uma função opcional **printarListaAdjacencia()** que descreve todas as entradas da lista de adjacência gerada.

A primeira etapa do código é criar duas variáveis:

- **char* arquivo**: uma string contendo o endereço do arquivo a ser lido;
- **int num_vertices**: inteiro referente ao número total de vértices + 1, servindo de limite para a os vetores posteriormente computados.

Ambas são utilizadas pela função **varreduraListaAdjacências()**, que atualiza o valor de **num_vertices** para o input de um arquivo sem antes conhecer o seu tamanho. Depois o grafo é criado por **novoGrafo()** e alimentado por

carregarListaAdjacencias(). As informações mais cruciais, como número de vértices, grau máximo e grau mínimo são mostradas pelo terminal.

```
10  ▾ int main(){
11      char *arquivo = "teste2.csv";
12
13      int num_vertices=0;
14
15      varreduraListaAdjacencias(arquivo, &num_vertices);
16
17      Grafo *grafo = novoGrafo(num_vertices);
18      carregarListaAdjacencias(grafo, arquivo);
19
20      printf("\n-----Dados sobre o Grafo-----\n\n");
21  ▾  printf("Quantidade de vertices:\t%d\nGrau maximo:\t%d\nGrau minimo:\t%d\n\n",
22      num_vertices-1, grauMaximo(grafo), grauMinimo(grafo));
23
```

Figura 2: print inicial e criação do grafo.

A Figura 3 mostra a função **ehMultigrafo()** e os dois valores inteiros atualizados com a sua chamada, os quais informam a quantidade de laços e arestas múltiplas, respectivamente, no caso do grafo ser um multigrafo.

```
24      int multigrafo_bool, lacos=0, repeticoes=0;
25      multigrafo_bool = ehMultigrafo(grafo, &lacos, &repeticoes);
26      printf("Eh um multigrafo? %s\n\n", multigrafo_bool ? "Sim." : "Nao.");
27      if(multigrafo_bool){
28          printf("\tLacos:\t\t%d\n\tArestas mult:\t%d\n\n", lacos, repeticoes);
29      }
```

Figura 3.

A Figura 4 mostra a descrição dos componentes conexos composta por 3 variáveis associadas à função **componentesConexos()**:

- **int num_componentes**: quantidade de componentes;
- **int* tamanhos**: quantidade de vértices em cada componente;
- **int** componentes**: matriz contendo a distribuição dos vértices em cada componente.

Naturalmente, por **adicionarAresta()** inserir cada nova aresta de modo ordenado no encadeamento dos vértices, os vértices de cada componente conexo também estarão em ordem crescente.

Por fins de visualização do grafo completo, criou-se também a função **printarListaAdjacencias()** para apresentar no terminal a lista encadeada com todos

os seus vértices separados por colchetes e flechas para simbolizar os blocos encadeados. E uma vez que 999 vértices facilmente poluem a visão do terminal, ela foi comentada para a visualização dos dados mais importantes.

```
31     int num_componentes=0;
32     int* tamanhos = (int*)calloc(grafo->num_vertices, sizeof(int));
33     int** componentes = componentesConexos(grafo, &num_componentes, tamanhos);
34
35     printf("-----Componentes Conexos do Grafo-----\n");
36     for(int i=0; i<num_componentes; i++){
37         printf("\nComponente %d de tamanho %d:\n", i+1, tamanhos[i]);
38         for(int j=0; j<tamanhos[i]; j++){
39             printf("%d ", componentes[i][j]);
40         }
41         printf("\n");
42     }
```

Figura 4.

3.2. Funções e Structs Criados

A seguir, serão apresentados o funcionamento e os objetivos das funções implementadas no arquivo **arq.h**, responsável pela maior parte da lógica e das funcionalidades principais do programa.

Na Figura 5 temos as linhas iniciais do arquivo **arq.h**

```
1     #ifndef ARQ_H
2     #define ARQ_H
3
4     #include <stdio.h>
5     #include <stdlib.h>
6     #include <string.h>
7
8     #define BUFFER 1024
```

Figura 5: linhas iniciais.

As linhas um e dois, são responsáveis em impedir que o mesmo arquivo **.h** seja incluído mais de uma vez no programa. Nas linhas 4, 5 e 6 temos as bibliotecas utilizadas. Foram carregadas as bibliotecas específicas **stdlib.h** e **string.h**, **stdlib.h** para **malloc**, **calloc**, **free**, entre outras, e **string.h** para **strtok**. O **define** da linha 8 cria uma constante chamada **BUFFER** com valor 1024 que representa o tamanho máximo do vetor de caracteres usado para leitura das linhas do arquivo.

```

10  typedef struct Vertice{
11      struct Vertice* anterior;
12      struct Vertice* posterior;
13      int valor;
14  } Nodo, Vertice;
15
16  /*
17      O grafo apresenta uma lista de adjacências composta por todos os vertices
18      encadeados ordenadamente com os vertices com arestas em comum..
19  */
20  typedef struct{
21      Vertice* lista;
22      int* graus;           // Graus de cada vertice.
23      int num_vertices;
24  } Grafo;

```

Figura 6: structs.

A *struct* **Vertice** representa os vértices e os nós da lista de adjacências do grafo. Há dois ponteiros, um para o elemento anterior e outro para o elemento posterior, e o valor que é utilizado para identificação do vértice. Já o *struct* **Grafo** é composto pela lista de adjacências, uma lista contendo os graus de cada vértice (endereço que ajuda na lógica de outras funções) e o número limite de vértices.

As funções **novoVertice()** e **novoGrafo()** são funções básicas para a alocação de memória de cada struct e seus componentes internos. Em **novoGrafo()**, **lista** é alocada dinamicamente e possui cada vértice seu iniciado com o valor de seu próprio índice, como demanda a estrutura de uma lista de adjacências. A Figura 7 mostra **inserirVertice()** e **inserirAresta()**, logo as principais funções para o encadeamento do grafo. Vale ressaltar que o código trabalha com grafos não-direcionados, ou seja, a aresta recíproca de cada aresta inserida também precisa ser considerada pelas operações que se utilizam da lista de adjacências.

carregarListaAdjacencias() e **varreduraListaAdjacencias()** possuem um início similar quanto à leitura do arquivo, porém **varreduraListaAdjacencias()** apenas se encarrega de retornar o maior valor inteiro lido no arquivo (lógica que, para arquivos similares a *teste2.csv*, é o equivalente a ler o total de linhas), logo maior vértice do grafo, a fim de servir como valor limite dos endereçamentos de vetores nas demais funções. Já **carregarListaAdjacencias()** se utiliza do *token* gerado pela função a partir das leituras de cada linha para incluir uma aresta por vez e estruturar o grafo.

```

62  */
63     "origem" se refere ao valor do grafo da lista de adjacencia, enquanto
64     "destino" serah o valor do novo vertice a ser encadeado.
65  */
66  void inserirVertice(Grafo* grafo, int origem, int destino){
67      Vertice* v_origem = &(grafo->lista[origem]);
68      Vertice* v_destino = novoVertice(destino);
69
70      // Caso o vertice de origem nao estiver encadeado a nenhum vertice.
71      if(v_origem->posterior == NULL){
72          v_destino->anterior = v_origem;
73          v_origem->posterior = v_destino;
74      }
75      else{
76          Vertice* walker = v_origem->posterior;
77          Vertice* v_anterior = v_origem;
78          while(walker != NULL && walker->valor < destino){
79              v_anterior = walker;
80              walker = walker->posterior;
81          }
82          v_destino->posterior = walker;
83          v_destino->anterior = v_anterior;
84          v_anterior->posterior = v_destino;
85      }
86      grafo->graus[origem]++;
87  }
88
89  */
90     Adiciona a aresta no vertice "origem" e a aresta reciproca para o vertice
91     "destino". Aqui eh desconsiderado o vertice 0 na interpreacao do grafo, mas
92     computacionalmente ele serah utilizado apenas como um indice da lista.
93  */
94  void adicionarAresta(Grafo* grafo, int origem, int destino){
95      if(origem <= 0 || destino <= 0){return;}
96
97      inserirVertice(grafo, origem, destino);
98
99      if(origem != destino){
100          inserirVertice(grafo, destino, origem);
101      }
102  }

```

Figura 7: Funções de encadeamento dos vértices.

```

143 void* *carregarListaAdjacencias(Grafo* grafo, char* path){
144     FILE* file = fopen(path, "r");
145     if(!file){
146         printf("Erro ao abrir o arquivo\n");
147     }
148
149     // Buffer para cada linha do arquivo e ignora a primeira linha (metadados?)
150     char buffer[BUFFER];
151     fgets(buffer, BUFFER, file);
152
153     while(fgets(buffer, BUFFER, file)){
154         char* token = strtok(buffer, " \n");
155
156         if(token == NULL){continue;}
157
158         /*
159          Guarda o primeiro vertice de uma linha em "origem" e adiciona na
160          linha de adjacencia correspondente os vertices associados.
161         */
162         int origem = strtol(token, NULL, 10);
163         token = strtok(NULL, " \n");
164         while(token != NULL){
165             adicionarAresta(grafo, origem, strtol(token, NULL, 10));
166             token = strtok(NULL, " \n");
167         }
168     }
169     fclose(file);
170 }

```

Figura 8: função de carregar lista de adjacências.

As funções **grauMaximo()** e **grauMinimo()** se utilizam do vetor **graus** em **Grafo** para fazer uma comparação simples e retornar o maior e menor valores do vetor, respectivamente. A função **ehMultigrafo()** (presente na figura 9) é responsável por verificar se o grafo é simples ou multigrafo. Um multigrafo é aquele que contém laços e/ou arestas múltiplas. Primeiro a função procura laços, que são arestas que se conectam no mesmo vértice, e depois são feitas comparações entre os elementos da lista de adjacências para identificar se há arestas múltiplas, ou seja, conexões repetidas entre dois vértices diferentes. Se forem encontrados laços ou arestas múltiplas, o grafo é classificado como multigrafo.

Para identificar componentes conexos utilizamos a busca em profundidade (DFS), e a função responsável é a **DFS_G()** como mostra na figura 10, a partir de um vértice inicial ela percorre todos os vértices conectados a ele, marcando eles como visitados para não haver repetições. Assim é possível separar os vértices de acordo com seus componentes conexos, e determinar a distribuição dos vértices em cada componente, a quantidade total de componentes conexos e o tamanho de cada componente encontrado.

A função **componentesConexos()** (Figura 11) é responsável por percorrer todos os vértices do grafo e iniciar uma nova DFS sempre que encontra um vértice ainda não visitado. Cada nova execução da DFS representa a descoberta de um novo componente conexo. Durante a busca, os vértices encontrados são armazenados em uma matriz, enquanto um vetor auxiliar registra o tamanho de cada componente identificado.

```
197 int ehMultigrafo(Grafo* grafo, int* lacos, int* repeticoes){
198     for(int i=1; i<grafo->num_vertices; i++){
199         Vertice* walker = grafo->lista[i].posterior;
200
201         while(walker != NULL){
202             if(walker->valor == i){
203                 (*lacos)++;
204             }
205             Vertice* skin = grafo->lista[i].posterior;
206
207             // Analogico a um for(){}, onde "walker" eh o limite de "skin".
208             while(walker != skin){
209                 if(walker->valor == skin->valor){
210                     (*repeticoes)++;
211                     break; // Na primeira nova ocorrencia de uma repeticao,
212                         // passa para o proximo walker e evita redundancia.
213                 }
214                 skin = skin->posterior;
215             }
216             walker = walker->posterior;
217         }
218     }
219     if(*lacos == 0 && *repeticoes == 0){return 0;}
220     return 1;
221 }
```

Figura 9: função para classificar grafo.

```
230 void DFS_G(Grafo* grafo, int raiz, int* vet_marca, int* tamanho, int* componente){
231     // Caso nao se queira retornar um vetor com os valores marcados.
232     if(vet_marca == NULL){
233         vet_marca = (int*)calloc(grafo->num_vertices, sizeof(int));
234     }
235     vet_marca[raiz] = 1;
236     // printf("%d ", raiz);
237     componente[*tamanho] = raiz;
238     (*tamanho)++;
239
240     Vertice* walker = grafo->lista[raiz].posterior;
241     while(walker != NULL){
242         if(vet_marca[walker->valor] == 0){
243             DFS_G(grafo, walker->valor, vet_marca, tamanho, componente);
244         }
245         walker = walker->posterior;
246     }
247 }
```

Figura 10: função DFS.

```
257 int** componentesConexos(Grafo* grafo, int* num_componentes, int* tamanhos){
258     // Distribuicao dos vertices por componentes conexos
259     int** componentes = (int**)calloc(grafo->num_vertices, sizeof(int*));
260     // Vetor principal de comparacao dos vertices
261     int* marcados = (int*)calloc(grafo->num_vertices, sizeof(int));
262     *num_componentes = 0;
263
264     for(int i=1; i<grafo->num_vertices; i++){
265         componentes[*num_componentes] = (int*)calloc(grafo->num_vertices, sizeof(int));
266         if(!marcados[i]){
267             tamanhos[*num_componentes] = 0;
268             DFS_G(grafo, i, marcados, &(tamanhos[*num_componentes]), componentes[*num_componentes]);
269             (*num_componentes)++;
270         }
271     }
272     return componentes;
273 }
```

Figura 11: função componentes conexos.

4. Dificuldades Encontradas

O enunciado da tarefa explicitava que o arquivo que deveria ser usado de teste e base para o código seria **teste2.csv**, composto por 999 linhas e 2 colunas somente, ou seja, o vértice respectivo à sua linha e apenas uma ou nenhuma adjacência. Até mesmo pela falta de especificação se este deveria ser o *único* arquivo a servir de base, foi-nos auto-imposto o desafio de generalizar os processos que antes seriam exclusivos para este teste para então servir para uma ampla gama de variações: praticamente qualquer número de linhas acima de 0; e linhas com

mais de uma adjacência, portanto mais de 2 colunas, sejam laços, arestas múltiplas ou várias adjacências diferentes entre si.

Esta decisão foi a origem da maioria das dificuldades encontradas, como por exemplo a leitura de uma linha do arquivo por ***strtk()*** ao invés de outra função que poderia ser mais simples considerando uma quantidade fixa máxima de números para extrair. Ou também as funções ***inserirVertice()*** e ***ehMultigrafo()*** que são partidas e preparadas para tratar um número indeterminado de vértices.

Também buscou-se resumir o máximo possível os algoritmos de cada função para que se declarasse o mínimo possível de variáveis, que as declaradas fossem aproveitadas ao máximo, e que fosse aplicada uma lógica simples e rápida. Como esperado, também se consumiu mais tempo aperfeiçoando as funções com base nestes pontos de modo que o código final seja mais “limpo” e direto, com comentários que ajudassem a explicá-lo.

5. Resultados

Abaixo segue o resumo dos dados extraídos pelo grafo gerado por ***teste2.csv*** de *input*, os quais também constam no *output* de ***main.c***, como mostra a Figura 11, além de todos os vértices que compõem ordenadamente cada componente:

- 1) Número de vértices: 999
- 2) Grau máximo: 2
- 3) Grau mínimo: 1
- 4) O grafo é simples, portanto não possui laços ou arestas.
- 5) O grafo possui 4 componentes conexos, sendo 3 deles com 250 vértices e 1 com 249 vértices (resultando em $3 \times 250 + 249 = 999$). O primeiro e quarto componentes possuem vértices ímpares, enquanto os demais possuem vértices pares, e cada um é dividido praticamente nos extremos entre cada quarto na divisão de $1000/4$.

```

guihreinert@Vindicaar:~/UDESC/4TEG/tarefa1$ ./main
-----Dados sobre o Grafo-----

Quantidade de vertices: 999
Grau maximo: 2
Grau minimo: 1

Eh um multigrafo? Nao.

-----Componentes Conexos do Grafo-----

Componente 1 de tamanho 250:
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55 57 59 61 63 65 67 69 71 73 75 77 79 81 83 85 87 89 91 93 95 97 99 101 103 105 107 109 111 113 115 117 119 121 123 125 127 129 131 133 135 137 139 141 143 145 147 149 151 153 155 157 159 161 163 165 167 169 171 173 175 177 179 181 183 185 187 189 191 193 195 197 199 201 203 205 207 209 211 213 215 217 219 221 223 225 227 229 231 233 235 237 239 241 243 245 247 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999

Componente 2 de tamanho 249:
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90 92 94 96 98 100 102 104 106 108 110 112 114 116 118 120 122 124 126 128 130 132 134 136 138 140 142 144 146 148 150 152 154 156 158 160 162 164 166 168 170 172 174 176 178 180 182 184 186 188 190 192 194 196 198 200 202 204 206 208 210 212 214 216 218 220 222 224 226 228 230 232 234 236 238 240 242 244 246 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999

Componente 3 de tamanho 250:
500 502 504 506 508 510 512 514 516 518 520 522 524 526 528 530 532 534 536 538 540 542 544 546 548 550 552 554 556 558 560 562 564 566 568 570 572 574 576 578 580 582 584 586 588 590 592 594 596 598 600 602 604 606 608 610 612 614 616 618 620 622 624 626 628 630 632 634 636 638 640 642 644 646 648 650 652 654 656 658 660 662 664 666 668 670 672 674 676 678 680 682 684 686 688 690 692 694 696 698 700 702 704 706 708 710 712 714 716 718 720 722 724 726 728 730 732 734 736 738 740 742 744 746 748 750 752 754 756 758 760 762 764 766 768 770 772 774 776 778 780 782 784 786 788 790 792 794 796 798 800 802 804 806 808 810 812 814 816 818 820 822 824 826 828 830 832 834 836 838 840 842 844 846 848 850 852 854 856 858 860 862 864 866 868 870 872 874 876 878 880 882 884 886 888 890 892 894 896 898 900 902 904 906 908 910 912 914 916 918 920 922 924 926 928 930 932 934 936 938 940 942 944 946 948 950 952 954 956 958 960 962 964 966 968 970 972 974 976 978 980 982 984 986 988 990 992 994 996 998 999

Componente 4 de tamanho 250:
501 503 505 507 509 511 513 515 517 519 521 523 525 527 529 531 533 535 537 539 541 543 545 547 549 551 553 555 557 559 561 563 565 567 569 571 573 575 577 579 581 583 585 587 589 591 593 595 597 599 601 603 605 607 609 611 613 615 617 619 621 623 625 627 629 631 633 635 637 639 641 643 645 647 649 651 653 655 657 659 661 663 665 667 669 671 673 675 677 679 681 683 685 687 689 691 693 695 697 699 701 703 705 707 709 711 713 715 717 719 721 723 725 727 729 731 733 735 737 739 741 743 745 747 749 751 753 755 757 759 761 763 765 767 769 771 773 775 777 779 781 783 785 787 789 791 793 795 797 799 801 803 805 807 809 811 813 815 817 819 821 823 825 827 829 831 833 835 837 839 841 843 845 847 849 851 853 855 857 859 861 863 865 867 869 871 873 875 877 879 881 883 885 887 889 891 893 895 897 899 901 903 905 907 909 911 913 915 917 919 921 923 925 927 929 931 933 935 937 939 941 943 945 947 949 951 953 955 957 959 961 963 965 967 969 971 973 975 977 979 981 983 985 987 989 991 993 995 997 999

```

Figura 11: Recorte do output de main.c carregando o arquivo teste2.csv.

Para facilitar a visualização e análise dos dados obtidos, foi produzido um histograma (Figura 12) representando os componentes conexos identificados no grafo.

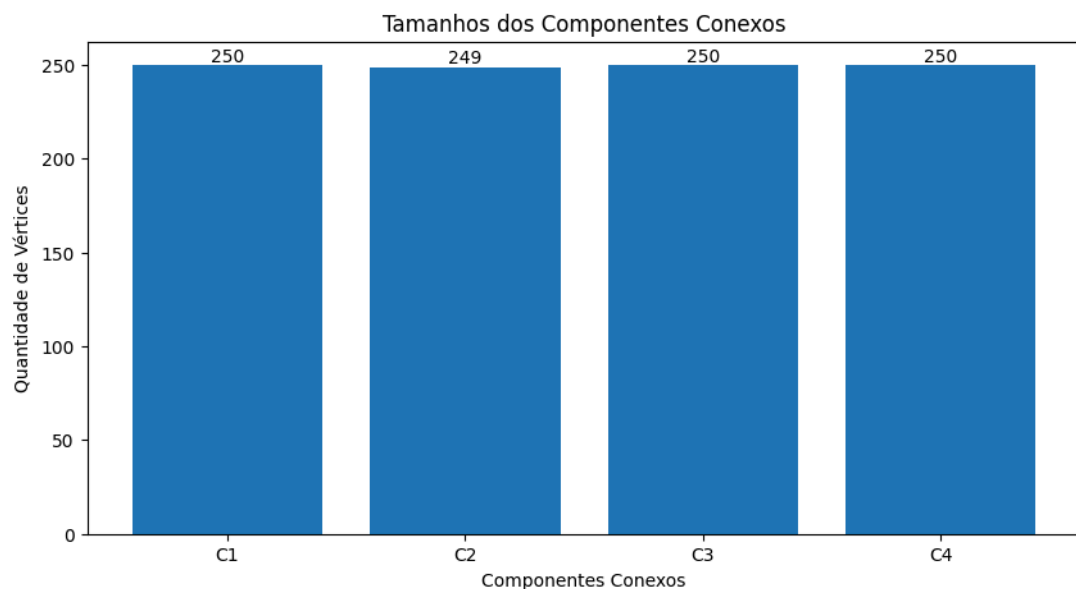


Figura 12: Histograma da distribuição dos componentes conexos.